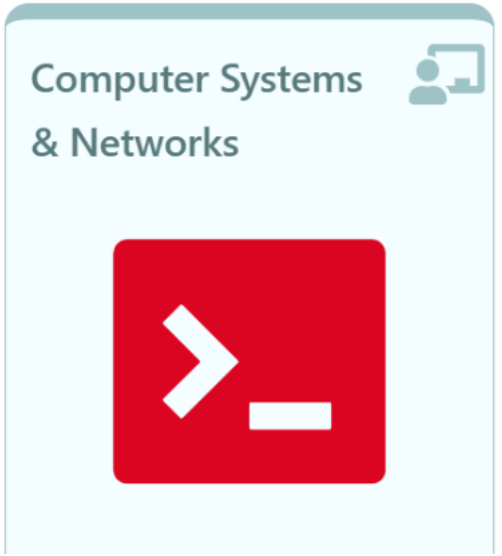
Computer Systems & Networks



SPRINGBOARD
www.springboardcourses.ie

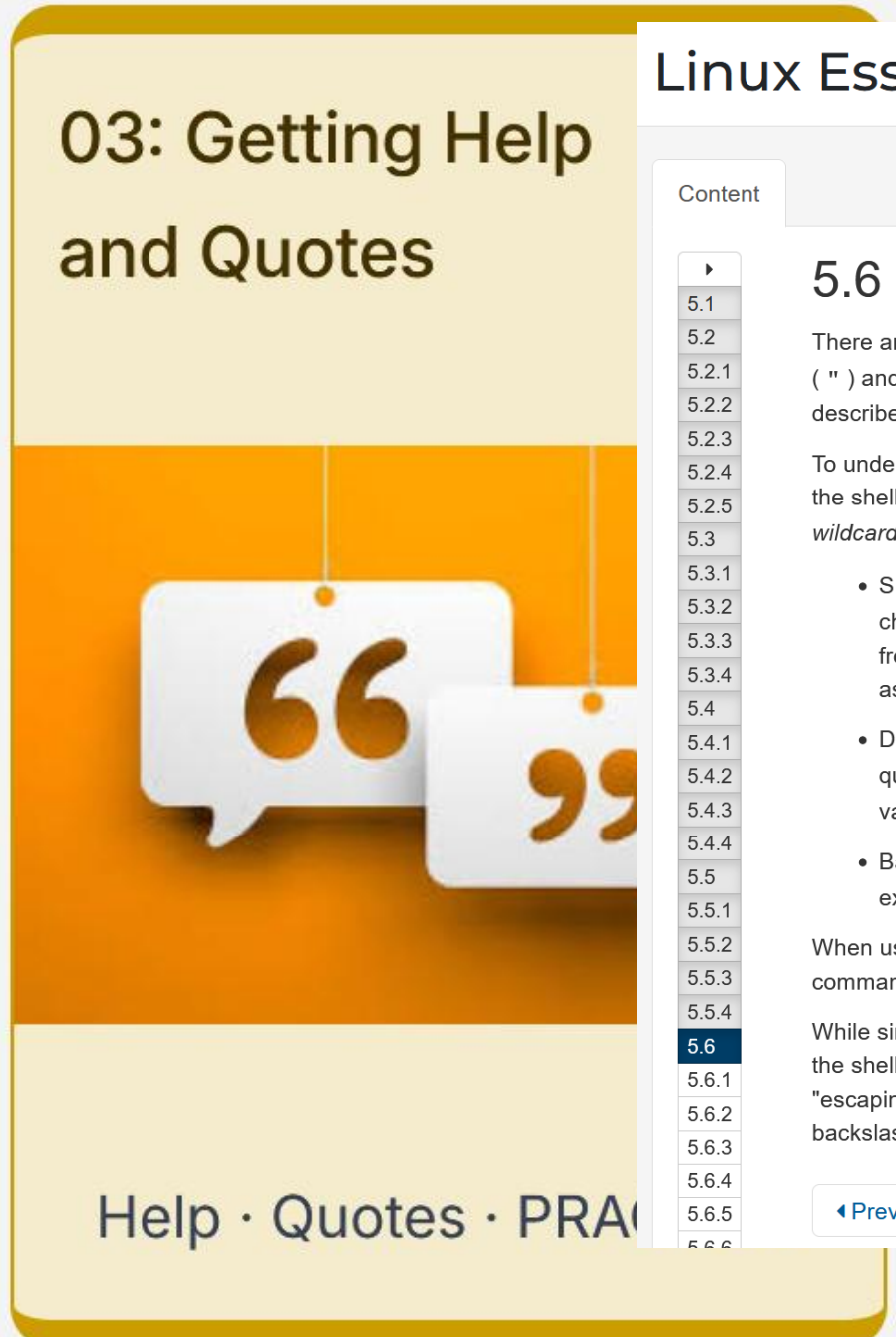
Ireland's European Structural and Investment Funds Programmes 2014-2020
Co-funded by the Irish Government and the European Union

OIDEACHAS AGUS EOLAÍOCHTA | EDUCATION AND SCIENCE

HEA | HIGHER EDUCATION AUTHORITY
AN tÚDARÁS um ARD-OIDEACHAS

European Union
Investing in your future
European Social Fund

SETU
Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University



Linux Essentials

Content

5.6 Quoting

There are three types of quotes used by the Bash shell: single quotes (`'`), double quotes (`"`) and back quotes (`` ` ``). These quotes have special features in the Bash shell as described below.

To understand single and double quotes, consider that there are times that you don't want the shell to treat some characters as *special*. For example, the `*` character is used as a *wildcard*. What if you wanted the `*` character to just mean a literal asterisk?

- Single `'` quotes prevent the shell from "interpreting" or expanding all special characters. Often single quotes are used to protect a string (a sequence of characters) from being changed by the shell, so that the string can be interpreted by a command as a parameter to affect the way the command is executed.
- Double `"` quotes stop the expansion of glob characters like the asterisk (`*`), question mark (`?`), and square brackets (`[]`). Double quotes *do* allow for both variable expansion and command substitution (see back quotes) to take place.
- Back `` ` `` quotes cause *command substitution* which allows for a command to be executed within the line of another command.

When using quotes, they must be entered in pairs or else the shell will not consider the command complete.

While single quotes are useful for blocking the shell from interpreting one or more characters, the shell also provides a way to block the interpretation of just a single character called "escaping" the character. To *escape* the special meaning of a shell metacharacter, the backslash `\` character is used as a prefix to that one character.

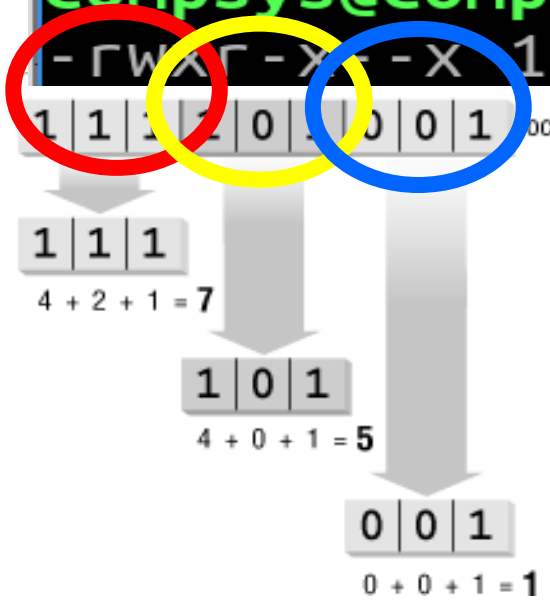
Previous Next

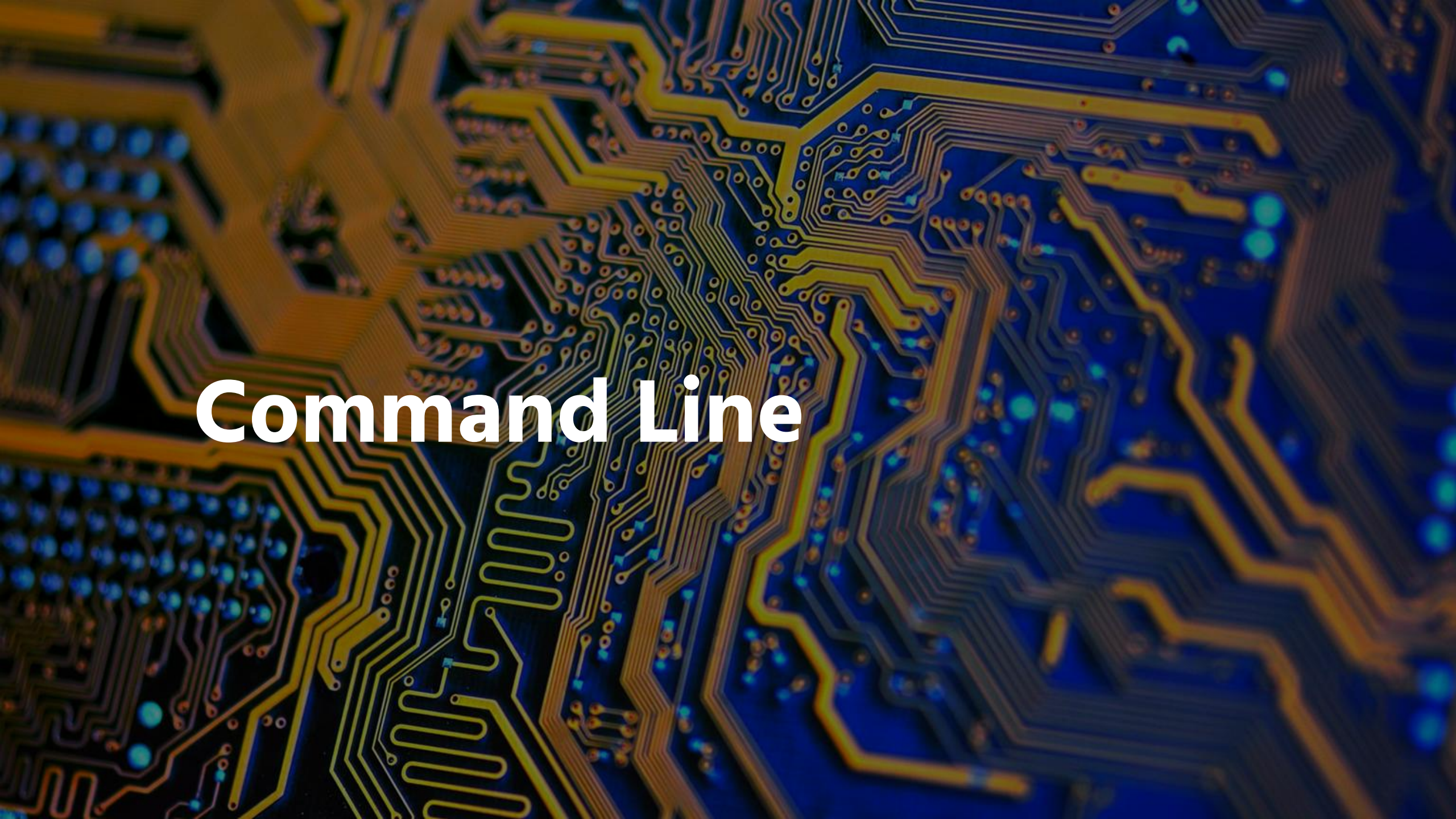
Week 1: Octal Recap

Octal Representation

0	000	-	-	-	No permissions
1	001	-	-	x	Only Execute
2	010	-	w	-	Only Write
3	011	-	w	x	Write and Execute
4	100	r	-	-	Only Read
5	101	r	-	x	Read and Execute
6	110	r	w	-	Read and Write
7	111	r	w	x	Read, Write and Execute

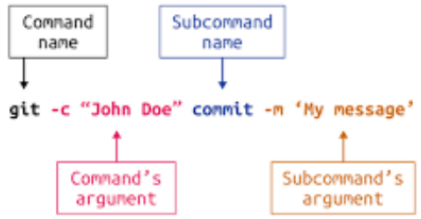
```
compsys@compsys-virtualbox:~$ chmod 751 stats
compsys@compsys-virtualbox:~$ ls -l st*
-rwxr-x--x 1 compsys compsys 248 Aug 29 13:54 stats
```




Command Line

Learning Outcomes today:

01: Command Line

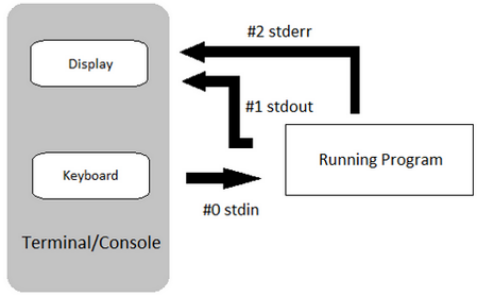


Commands · Arguments · Options · Variables

02: File Management

Globs · Copy · Move · Remove

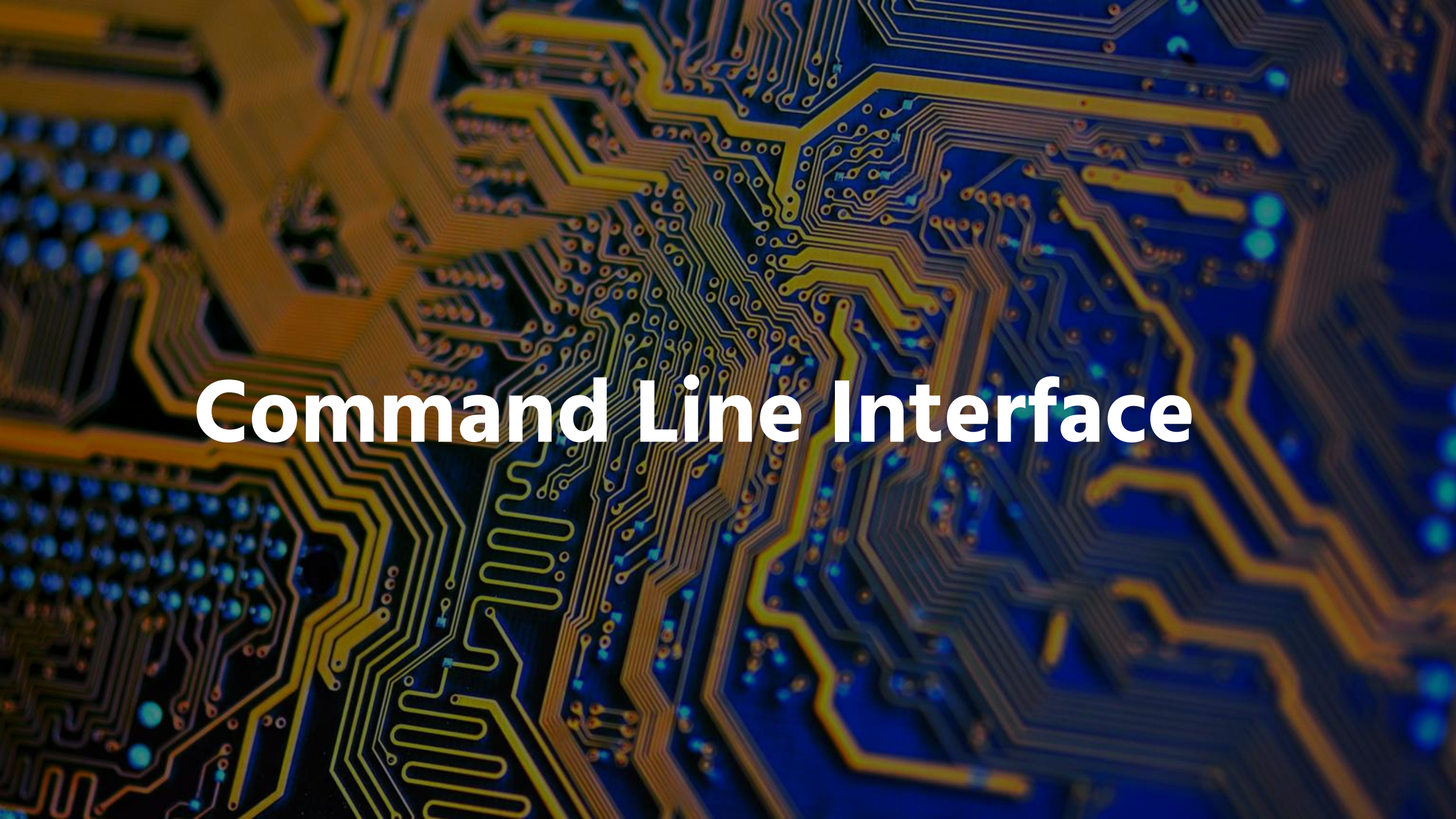
03: IO Redirection



STDIN · STDOUT · STDERR

Lab

File Management · IO Redirection · Search & Filter text


Command Line Interface

Command Line Interface

The Linux community promotes the CLI due to its power, speed and ability to accomplish a vast array of tasks with a **single command line** instruction.

The CLI provides

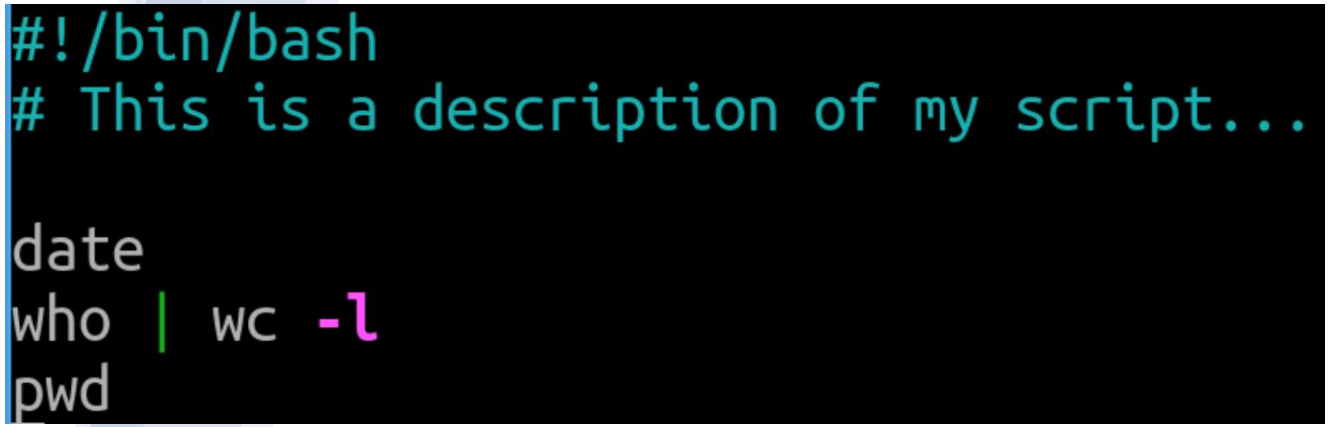
- more precise control,
- greater speed and
- the ability to automate tasks more easily through scripting.

The Shell

The Shell

Once you've entered a command the terminal,
- The terminal then accepts the command & passes to a *shell*.

The shell is the CLI that translates commands entered by a user into **actions** to be performed by the OS.



The Shell

The Bash shell also has many popular features, such as:

- Command line history
- Inline editing
- Scripting
  - The ability to place commands in a file and then interpret (effectively use Bash to execute the contents of) the file, resulting in all the commands being executed.
- Aliases
  - The ability to create short nicknames for longer commands.
- Variables
  - Used to store information for the Bash shell and for the user.

The Shell

Typically the prompt contains information about the user and the system. Below is a common prompt structure:

A typical prompt shown contains the following information:

Username (sysadmin)	System name (localhost)	Current Directory (~)


Commands

Commands

```
compsys@compsys-virtualbox:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

A command is a software program that when executed on the CLI, performs an action on the computer.

To execute a command, the first step is to type the name of the command.

If you type ls and hit **Enter**. The result should resemble the example below:

Commands

```
command [options] [arguments]
```

Some commands require additional input to run correctly.

This additional input comes in two forms: *options* and *arguments*.

- **Options** are used to modify the core behavior of a command.
- **Arguments** are used to provide additional information (such as a filename or a username).


Options

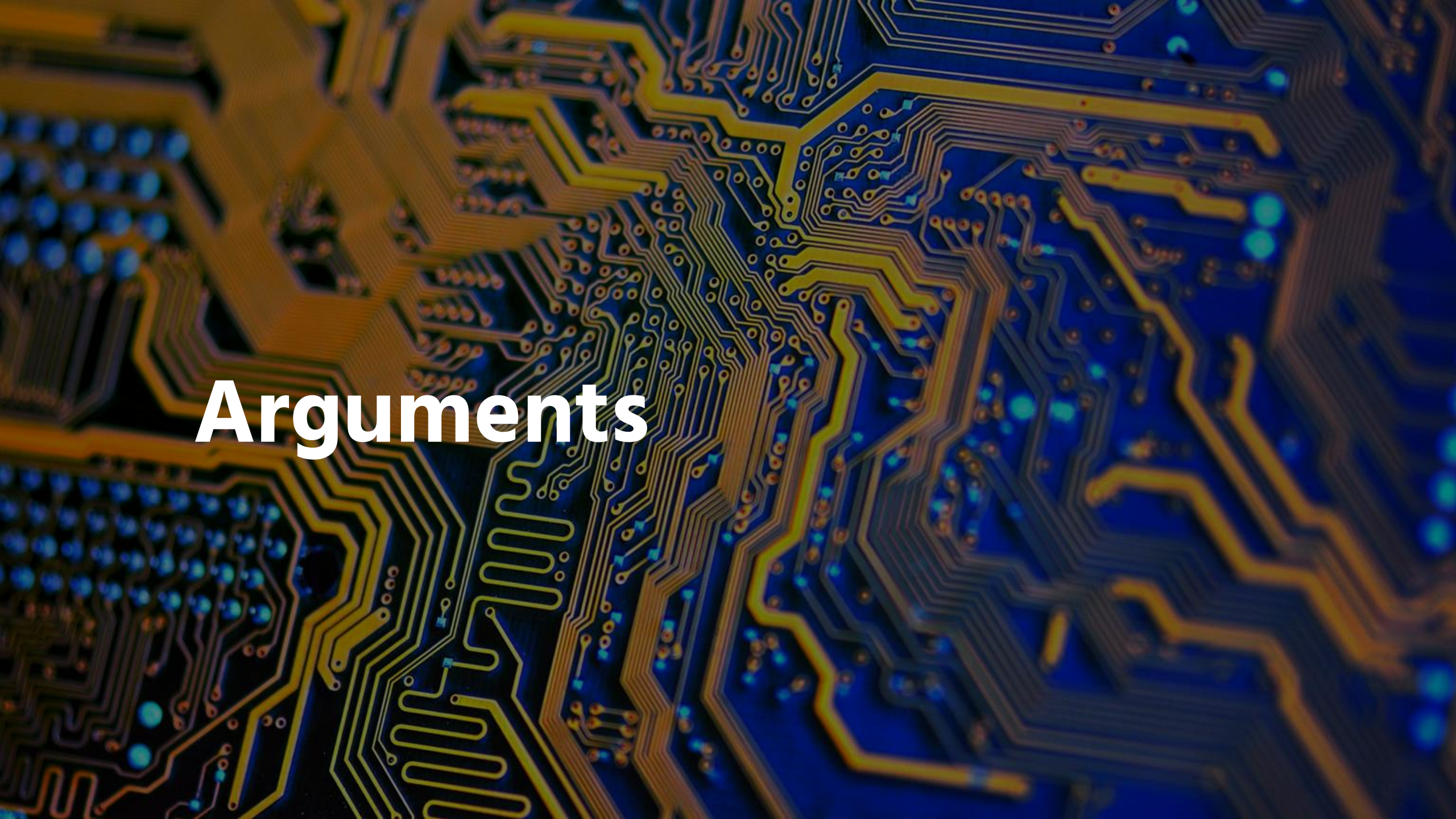
Options

```
command [options] [arguments]
```

used with commands to expand or modify the way a command behaves.

```
compsys@compsys-virtualbox:~$ ls -l
total 40
drwxr-xr-x 2 compsys compsys 4096 Sept 29  20:21 Desktop
drwxr-xr-x 1 compsys compsys 4096 Sept 29  20:25 Documents
```

```
compsys@compsys-virtualbox:~$ ls -lr
```


Arguments

Arguments

```
command [options] [arguments]
```

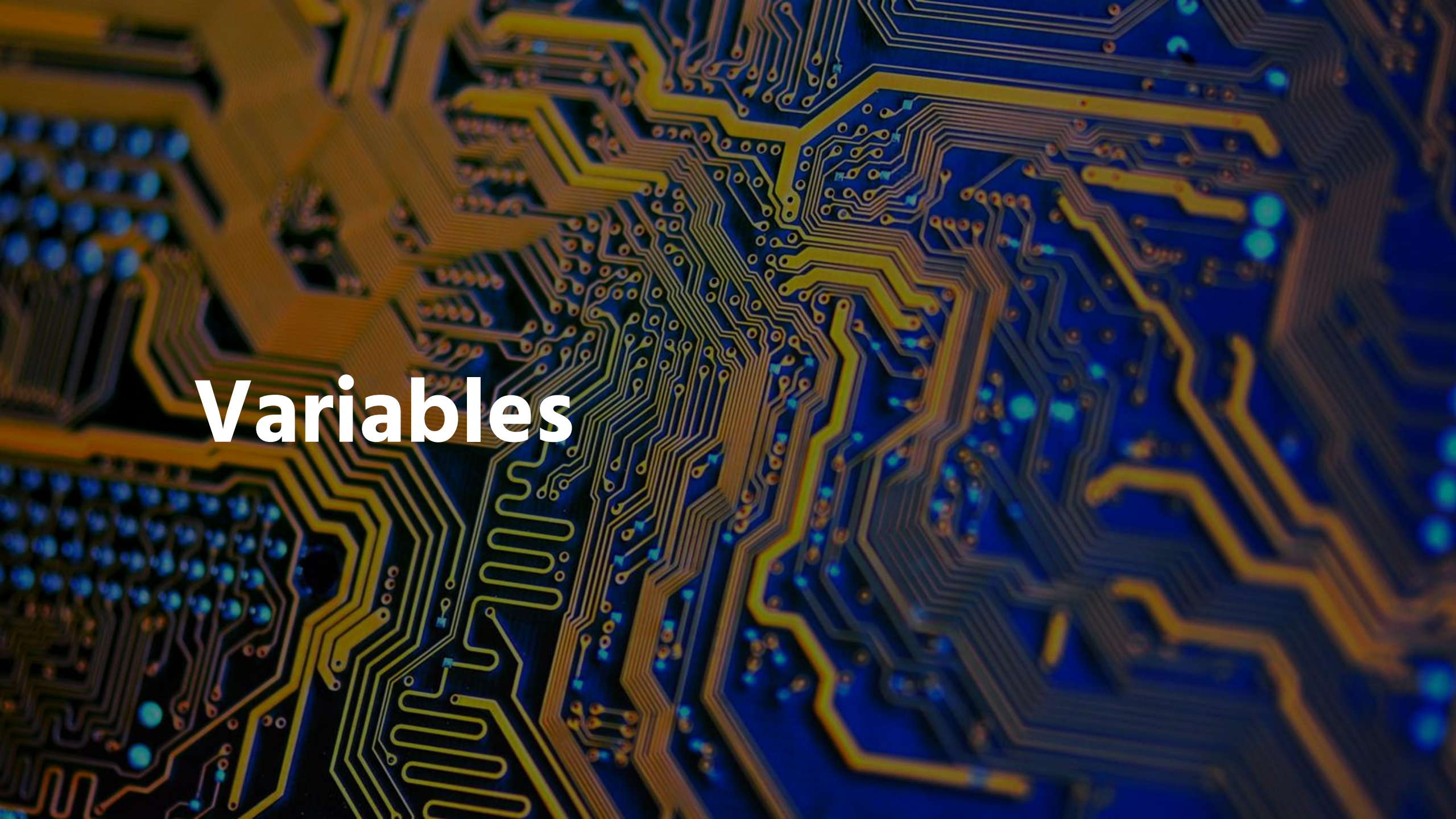
… used to specify something for the command to act upon.

If the ls command is given the name of a directory as an argument, it lists the contents of that directory

Some commands (such as ls) accept **multiple** arguments

```
compsys@compsys-virtualbox:~$ ls /etc/ppp
ip-down.d  ip-up.d
```

```
compsys@compsys-virtualbox:~$ ls /etc/ppp /etc/ssh
```


Variables

Variables

A variable is a feature that allows the user or the shell to **store** data.

Variables are given names and stored temporarily in memory.

There are two types of variables used in the Bash shell,

1. local and *2. environmental*

1. Local Variables

Local or *shell* variables **exist only in the current shell:**

- When the user closes a terminal window or shell, all variables are lost.

Syntax: *variable=value*

```
compsys@compsys-virtualbox:~$ name="Thunderbirds"
compsys@compsys-virtualbox:~$ message="Welcome to $name are go!"
compsys@compsys-virtualbox:~$ echo "$message"
```

Display the value of the variable using a dollar sign $ character followed by the variable name as an argument to the echo command:

2. Environmental Variables

Environment variables are:

- named values that store information the OS & programs can use.
- are available system-wide

Examples include :

PATH (Where the shell looks for commands),
HOME (Your home directory),
USER (your username) and
HISTSIZE variables.

```
compsys@compsys-virtualbox:~$ echo $HISTSIZE
1000
compsys@compsys-virtualbox:~$
```

Linux Essentials

Content

5.4.2 Step 2

Environment variables are available system-wide. The system automatically recreates environment variables when a new shell is opened. Examples include the `PATH`, `HOME`, and `HISTSIZE` variables. The `HISTSIZE` variable defines how many previous commands to store in the history list. In the example below, the command will display the value of the `HISTSIZE` variable:

```
sysadmin@localhost:~$ echo $HISTSIZE
1000
sysadmin@localhost:~$
```

Previous

Next

Path Variable

The **PATH variable** is one of the most important environment variables.

- PATH tells the shell **where to look for programs** when you type a command.

```
compsys@compsys-virtualbox:~$ ls
```

→The shell looks in each directory listed in PATH, in order.
→When it finds the program `ls`, it runs it
→If it doesn't find it, you get: `command not found`

EXER: Display the path of the current shell:

```
compsys@compsys-virtualbox:~$ echo $PATH
/home/compsys/.config/nvm/versions/node/v14.3.0/bin:/usr/local/sbin:/usr/local/bin:/us
r/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
```

5.4.3 Step 3

Type the following command to display the value of the `PATH` variable:

```
echo $PATH
```

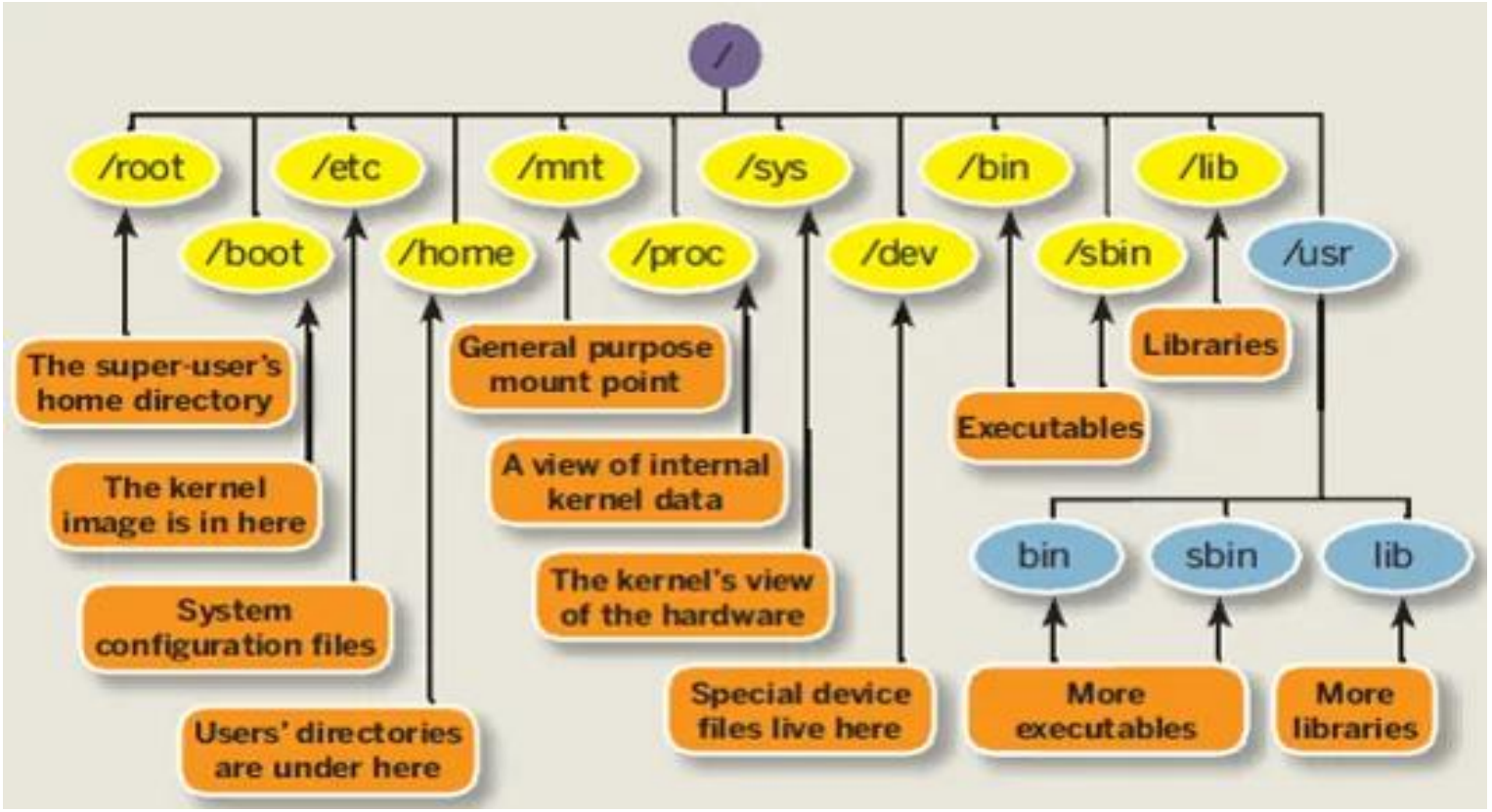
Your output should be similar to the following:

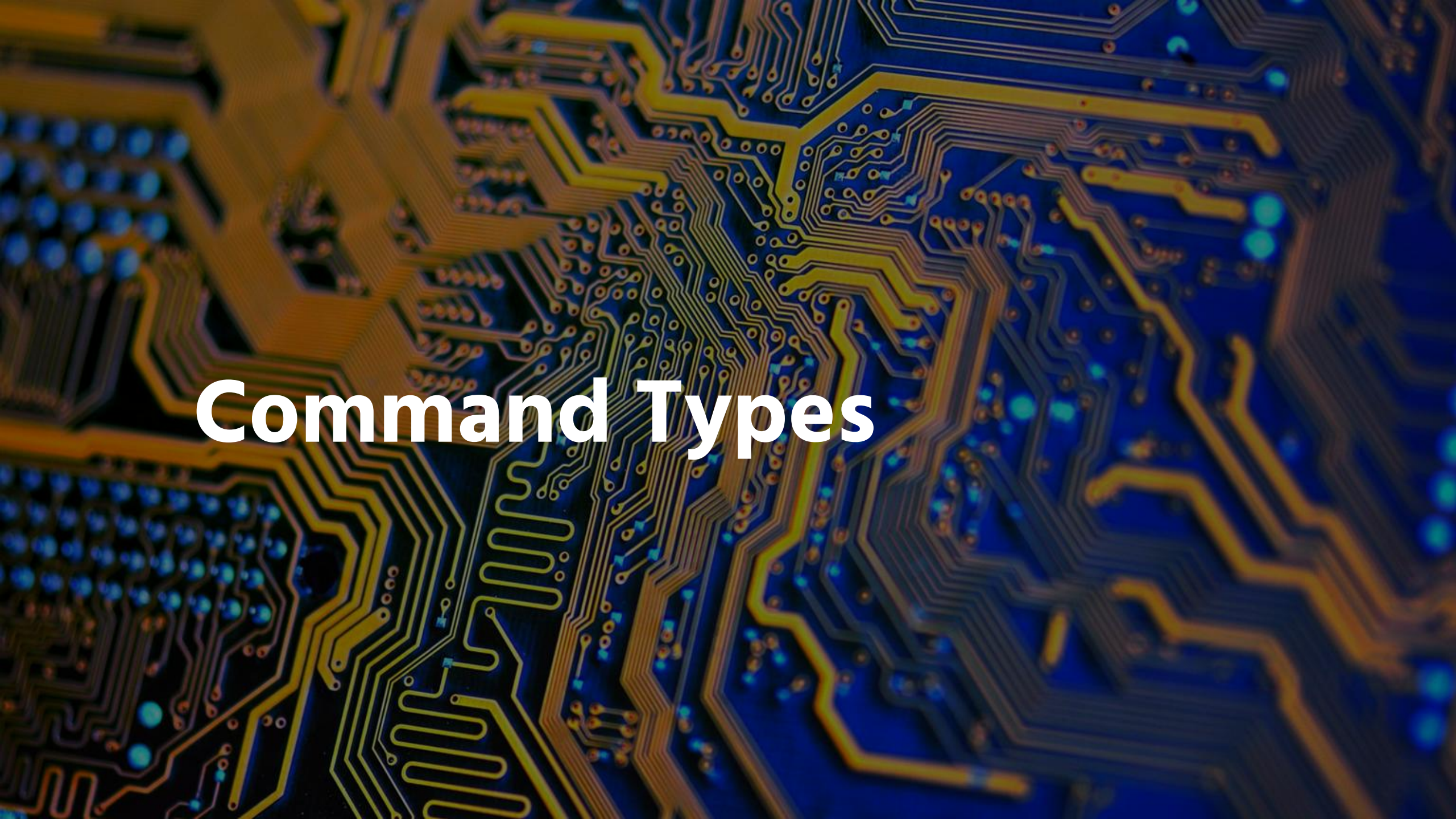
```
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr
/usr/games:/usr/local/games:/snap/bin
sysadmin@localhost:~$
```

The `PATH` variable is displayed by placing a `$` character in front of the name of the variable.

This variable is used to find the location of commands. Each of the directories listed above are searched when you run a command. For example, if you try to run the `date` command, the shell will first look for the command in the `/home/sysadmin/bin` directory and then in the `/usr/local/sbin` directory and so on. Once the `date` command is found, the shell "runs it".

Filesystem Hierarchy Structure (FHS)




Command Types

Command Types

The type command can be used to determine information about command type.

There are several different sources of commands within the shell of your CLI:

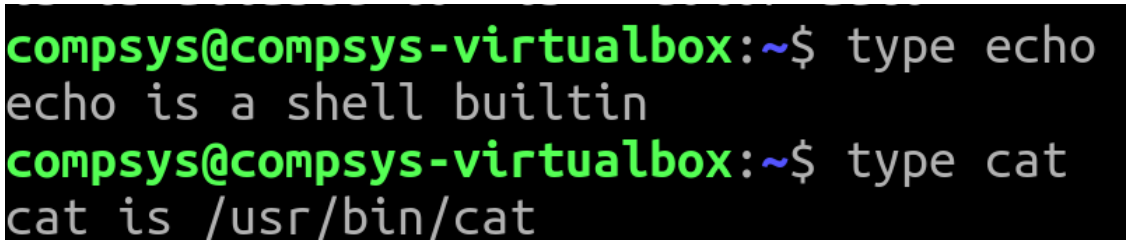
Internal commands	External commands	Aliases	Functions

Internal (built-in) Commands

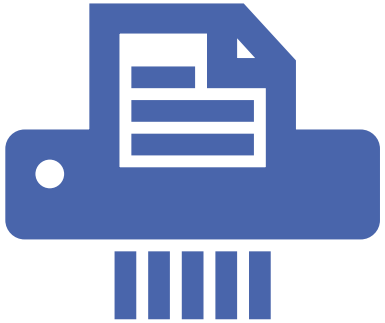
- Also called *built-in* commands, these commands are built into the shell itself.

  `cd` (change directory) command as it is part of the Bash shell.

- The `type` command identifies the `cd` command as an internal command:



External Commands

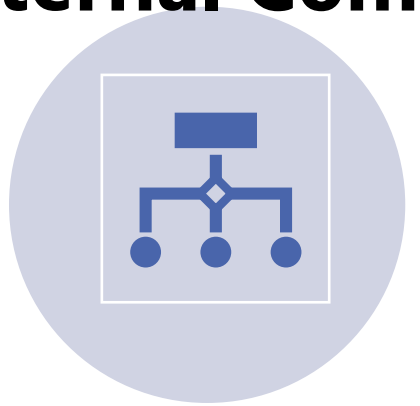


External commands are stored in files that are searched by the shell.

The which command searches for the **location** of a command by searching the PATH variable.

```
compsys@compsys-virtualbox:~$ which ls
/usr/bin/ls
```

External Commands



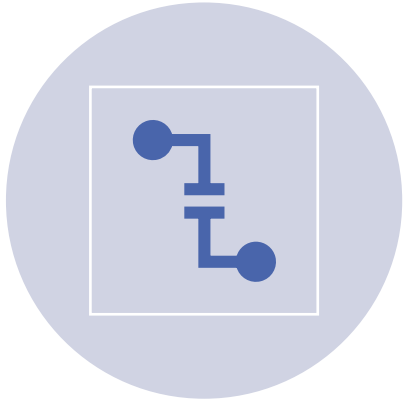
EXTERNAL COMMANDS CAN BE EXECUTED BY TYPING THE COMPLETE PATH TO THE COMMAND.

```
compsys@compsys-virtualbox:~$ /bin/ls

Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

FOR EXTERNAL COMMANDS, THE TYPE COMMAND DISPLAYS THE LOCATION OF THE COMMAND:

```
compsys@compsys-virtualbox:~$ type mkdir
mkdir is /usr/bin/mkdir
```



TO DISPLAY ALL LOCATIONS THAT CONTAIN THE COMMAND NAME, USE THE -A OPTION TO THE TYPE COMMAND:

```
ox:~$ type -a echo
```

Aliases

- An *alias* can be used to map longer commands to shorter key sequences.

- To determine what aliases are set on the current shell use the `alias` command:

```
compsys@compsys-virtualbox:~$ alias
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] &&
)" "$(history|tail -n1|sed -e '\''s/^\s*[0-9]\+\s*//;s/[;&
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -alF'
alias ls='ls --color=auto'
```

- The `type` command can identify aliases to other commands:

```
compsys@compsys-virtualbox:~$ type ll
ll is aliased to `ls -alF'
```

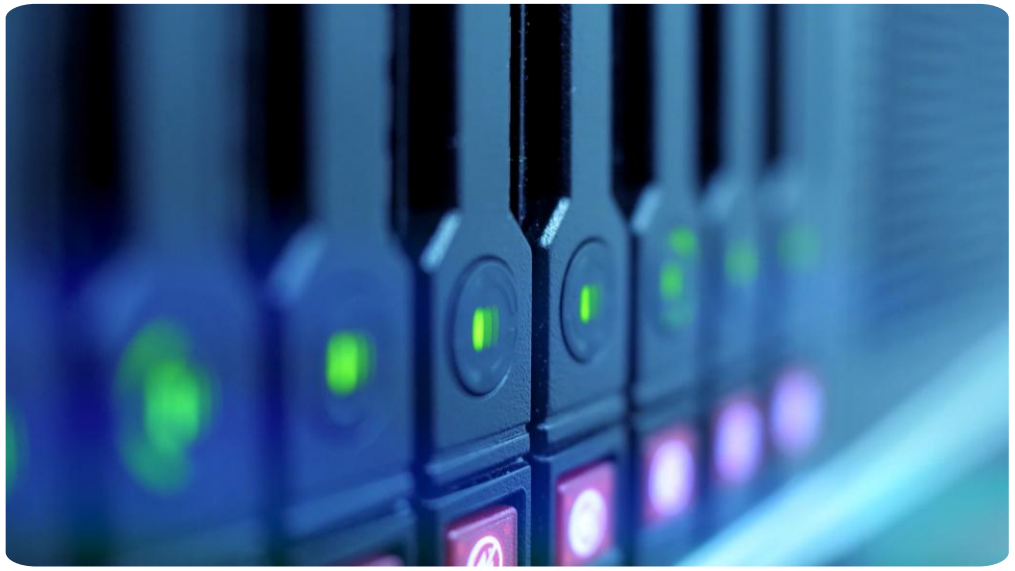
5.5.3 Step 3

Aliases can be used to map longer commands to shorter key sequences. When the shell sees an alias being executed, it substitutes the longer sequence before proceeding to interpret commands.

To determine what aliases are set on the current shell, use the `alias` command:

```
sysadmin@localhost:~$ alias
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo
 error)" "$(history|tail -n1|sed -e '\''s/^\s*[0-9]\+\s*//;s/[;&
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -alF'
alias ls='ls --color=auto'
```

Functions



- Functions can also be built using existing commands to:
  - Create new commands
  - Override commands built-in to the shell or commands stored in files